Learning Actionable Manipulation Recovery via Counterfactual Failure Synthesis

Dayou Li[1], Jiuzhou Lei[1], Hao Wang[1], Lulin Liu[1,2], Yunhao Yang[3], Zihan Wang[4], Bangya Liu[5]
Minghui Zheng[1], Zhiwen Fan[1]
[1]Texas A&M University, [2]University of Minnesota, [3]University of Texas at Austin
[4]Abaka AI, [5]University of Wisconsin-Madison

***Abstract*— While recent foundation models have significantly advanced robotic manipulation, these systems still struggle to autonomously recover from execution errors. Current failure-learning paradigms rely on either costly and unsafe real-world data collection or simulator-based perturbations, which introduce a severe sim-to-real gap. Furthermore, existing visual analyzers predominantly output coarse, binary diagnoses rather than the executable, trajectory-level corrections required for actual recovery. To bridge the gap between failure diagnosis and actionable recovery, we introduce Dream2Fix, a framework that synthesizes photorealistic, counterfactual failure rollouts directly from successful real-world demonstrations. By perturbing actions within a generative world model, Dream2Fix creates paired failure-correction data without relying on simulators. To ensure the generated data is physically viable for robot learning, we implement a structured verification mechanism that strictly filters rollouts for task validity, visual coherence, and kinematic safety. This engine produces a high-fidelity dataset of over 120k paired samples. Using this dataset, we fine-tune a vision-language model to jointly predict failure types and precise recovery trajectories, mapping visual anomalies directly to corrective actions. Extensive real-world robotic experiments show our approach achieves state-of-the-art correction accuracy, improving from 19.7% to 81.3% over prior baselines, and successfully enables zero-shot closed-loop failure recovery in physical deployments.**

I. Introduction

Recent foundation models, including vision-language models (VLMs) and vision-language-action models (VLAs), have introduced open-vocabulary perception and strong generalization to robotic manipulation [1]–[5]. Despite these advances, such models struggle to autonomously recover from real-world execution errors. While they excel at nominal task execution, they lack the causal understanding of failure dynamics required to synthesize executable corrective actions. Autonomous recovery necessitates not just detecting an anomaly, but reasoning about its root cause and translating that diagnosis into precise trajectory corrections. Developing this capability is severely bottlenecked by data; real-world failures rarely occur with paired, dense correction annotations, and intentionally collecting physical failure data across diverse tasks is both costly and hazardous to hardware.

Prior efforts to endow foundation models with failure-recovery capabilities largely follow two paradigms. The first utilizes reward signals or preference annotations to refine policies [6]–[8]. However, these methods frequently rely on video-level temporal heuristics that yield low-fidelity supervision, or depend on non-scalable human-in-the-loop feedback. The second paradigm scales data collection through procedural perturbations in simulation, training VLMs to output failure taxonomies [9]. While scalable, simulation-based generation suffers from a pronounced sim-to-real gap. Furthermore, these visual analyzers typically provide coarse, binary diagnoses rather than the low-level, executable trajectory corrections necessary for closed-loop recovery. Consequently, a persistent gap remains between high-level failure detection and actionable recovery (as shown in Tab. I).

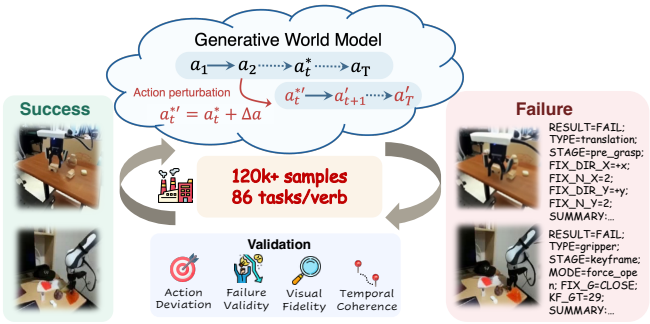


Fig. 1: Dream2Fix is a data generation pipeline that synthesizes large-scale, photorealistic failure rollouts with paired corrections from successful demonstrations and curates them with physical and visual verification.

To address these limitations, we introduce Dream2Fix, a data generation framework that synthesizes large-scale, photorealistic, and physically plausible counterfactual failure rollouts directly from successful real-world demonstrations (Fig. 1). By injecting targeted action perturbations (e.g., end-effector pose offsets) into a generative world model, Dream2Fix produces diverse failure videos paired with deterministic trajectory-level corrections. Because generative models can introduce visual or physical artifacts, we implement a structured verification mechanism. This verifier strictly filters synthesized samples across three dimensions: task failure validity, visual coherence, and kinematic safety. This ensures that retained rollouts accurately reflect the intended failure mode and yield executable corrective motions within robot constraints.

Using this engine, we curate a high-fidelity dataset of over 120,000 paired failure-correction samples, simultaneously mitigating the sim-to-real gap and providing denser super-

vision than prior datasets. We formulate failure recovery as a spatial reasoning task and fine-tune a VLM on this dataset to jointly predict failure diagnoses and precise trajectory corrections, directly mapping visual anomalies to recovery actions. Extensive evaluations demonstrate that our approach significantly improves correction accuracy over established baselines, from 19.7% to 81.3%, and achieves a 46% zero-shot recovery rate during real-robot closed-loop deployment.

To summarize, our main contributions are:

- We propose Dream2Fix, a data generation pipeline that leverages a generative world model to synthesize photorealistic robotic failure cases from successful demonstrations, governed by a rigorous physical and visual verifier.
- We introduce a failure-aware VLM that bridges diagnosis and actionable recovery by jointly predicting structured failure causes and precise, trajectory-level corrections.
- We establish a comprehensive evaluation framework for failure reasoning, demonstrating significant performance gains in both offline benchmarks and zero-shot, closed-loop real-robot recovery.

II. Related Works

A. Failure Detection in Robotic Manipulation

Failure in robotic manipulation is a common and critical challenge, and recent work has explored both detecting failure events and reasoning about their underlying causes. Gu et al. [10] analyze VLA feature representations to predict task failure likelihood, but do not provide corrective actions. Pacaud et al. [11] combine a simulator and limited real-robot perturbations to train a VLM for failure reasoning, but the real-world supervision is weak and misses execution-level failures. Liu et al. [12] collect failures from both simulation and real robots, but the dataset is small and limited in task and environment diversity. Qi et al. [13] use iterative self-refinement for failure detection and reasoning, but still rely on limited annotations and lack structured corrective supervision. Overall, prior works provide coarse failure reasoning but lacks actionable trajectory-level corrections, and remains constrained by unrealistic simulations or small real-world datasets, which makes it hard to generalize across diverse manipulation settings.

B. Data Generation in Robotics

A growing number of systems focus on scaling the collection of robotic data for training manipulation policies. Simulation-based pipelines have been widely adopted, such as MimicGen [14], SAGE [15], and RoboGen [16], which enable large-scale data collection and offer fine-grained control over task and environment. However, since they are conducted entirely in simulation, the resulting data often lacks visual realism, which brings large sim2real gap. Recent progress in world models suggests an alternative direction for generating realistic, action-conditioned data at scale [17]–[21], though ensuring physical plausibility remains challenging. These trends motivate leveraging world models to synthesize realistic failure data while incorporating explicit validity assessments to mitigate unrealistic generations.

C. Robotic Vision-Language Models

Recent robotic VLMs improve spatial grounding for manipulation, but they are mostly designed for perception and planning rather than failure-driven correction. SpatialBot [22] uses RGB-D supervision and hierarchical spatial Q&A to strengthen depth-aware understanding, but it does not learn structured recovery targets from failures. RoboRefer [23] resolves complex referential expressions via multi-stage spatial reasoning, but it does not support failure detection or recovery. RoboPoint [24] predicts dense affordance maps for where actions are feasible, yet it offers limited trajectory-level guidance for correcting execution errors.

III. Method

A. Problem Statement

We consider the task of diagnosing and correcting robotic manipulation failures from visual observations. Formally, given an input video sequence $\mathcal{I} = \{I_0, I_1, \ldots, I_T\}$, where $I_t \in \mathbb{R}^{H \times W \times 3}$ is an RGB frame at timestep t, the objective is to predict three outputs:

- **Result Classification:** a categorical label $y_{\text{result}} \in \{\text{success}, \text{fail}\}$ indicating the outcome of the manipulation attempt.
- **Failure Type:** a categorical label $y_{\text{fail}} \in \mathcal{F}$, where $\mathcal{F} = \{\text{translation}, \text{weak_close}, \text{force_open}, \text{delay_close}\}$ defines a set of failure modes.
- **Actionable Correction** a structured textual output $y_{\text{fix}} \in \mathcal{T}$ consisting of correction directives and a natural language summary. The directive fields specify how to correct the failure, including gripper actions at the keyframe or spatial adjustments.

We formulate failure recovery as structured prediction and fine-tune a VLM to jointly output success/failure, failure types, and trajectory-level recovery under a fixed schema. This structured supervision makes corrections interpretable and directly usable by robot controllers (as shown in Fig. 2).

B. Photorealistic Failure Generation

1) Failure Type Definition and Rollout Collection: To construct a dataset of diverse and interpretable failure cases, we define four representative failure types in robotic manipulation: delay_close, weak_close, force_open, and translation. These failure modes reflect common execution errors related to gripper control and spatial accuracy. Specifically:

- delay_close: the gripper closes with a significant delay relative to the intended grasping moment, resulting in a late or missed closure.
- weak_close: the gripper closes too softly or with incomplete force, which often leads to unstable contact and potential object drop.
- force_open: the gripper fails to close entirely at the designated keyframes, remaining open due to prediction errors by robotic policy.

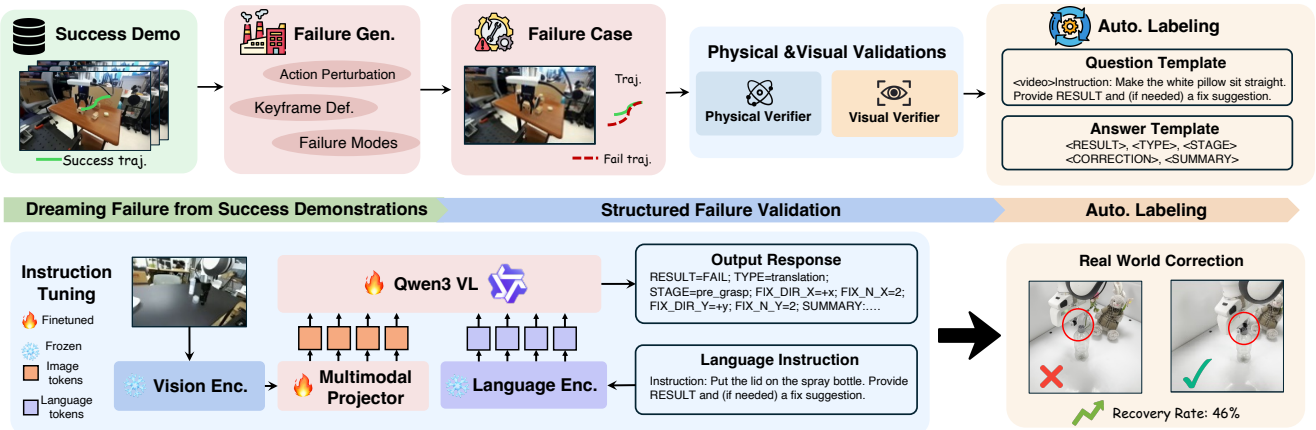


Fig. 2: **Overview of Dream2Fix pipeline**. Dream2Fix generates diverse failure cases from successful demonstrations via keyframe-level action perturbations, then validates and curates them with physical and visual verifiers. The verified rollouts are auto-labeled into a structured schema to instruction-tune a VLM that predicts actionable corrections for real-world recovery.

- translation: perturbations in the end-effector's x-y states during the approach and contact phase, often cause position misalignment or grasp failure

These types were chosen based on the subset of failure modes that can be stably induced and reproduced using current robotic world models. We adopt Ctrl-World [19] as the base generator. Unlike prior models that take in task descriptions, Ctrl-World is directly conditioned on robot actions, which we perturb to synthesize failure cases.

We define the keyframe as the timestep at which the gripper transitions between open and close states. For each rollout, we perturb actions over a local temporal window around the keyframes, thereby yielding more continuous rollouts and a higher probability of the desired failure. For instance, we add temporal offsets to delay gripper closure (delay_close), reduce the closure magnitude for weak_close, or invert the gripper action for force_open. For translation failures, we introduce Gaussian noise to the end-effector's x and y states near. These perturbations are minimal but sufficient to produce visible and meaningful deviations in the final rollout. Each video is annotated with a result label, a failure type, and a correction in structured text format. This strategy enables scalable and consistent synthesis of realistic failure data without any real world cost.

2) Structured Failure Validation: To ensure the physical plausibility and visual quality of the synthesized failures, we employ four verifiers to screen generated samples along four validation dimensions: task failure validity, visual quality (including temporal coherence), dynamics consistency, and kinematic safety. A sample is retained only if it satisfies all verifier conditions. We next describe the design and implementation of each verifier. A generated sample is retained only if it satisfies the corresponding validation criteria. In the following, we elaborate each verifier.

VLM Verifier. We design a VLM-based verifier to jointly assess whether a generated failure clip (i) truly manifests the intended failure and (ii) maintains adequate visual quality without synthesis artifacts or distortions. Concretely, we apply Qwen3-VL-8B as the base, together with the task instruction, the reference success clip, and the generated failure clip, to provide two judgments: task failure validity (whether the generated execution should be considered a failure compared to the reference) and visual quality (whether the video contains severe hallucinations). We retain a synthesized sample only when it is judged as a valid failure and passes the visual-quality criterion.

Inverse Dynamics Model (IDM) Verifier. We first train an IDM on successful demonstrations to capture the domain-specific action–state relationship. We instantiate the IDM with a ResNet-50 [25] visual backbone followed by an MLP head that regresses the state difference between two frames. Given two frames I_t and I_{t+d} from a generated rollout, the IDM f_θ infers the robotic end-effector's state difference implied by the visual transition:

$$\widehat{\Delta s}_{t,d} = f_\theta(I_t, I_{t+d}) \tag{1}$$

Each generated clip is synthesized conditioned on the recorded state s_t and action sequence, from which we compute the reference state difference

$$\Delta s_{t,d} = s_{t+d} - s_t \tag{2}$$

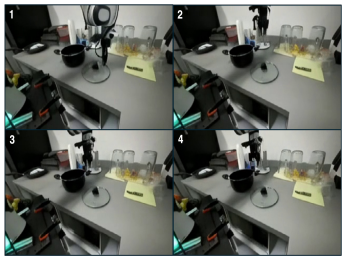
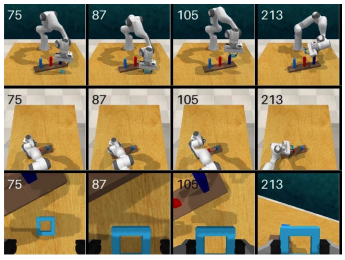
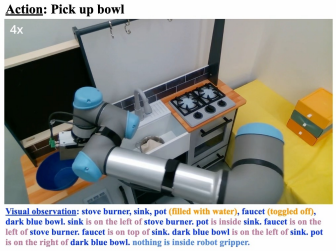
We measure the discrepancy between the visually inferred transition and the conditioning signal via

$$e_{t,d} = \left\| \widehat{\Delta s}_{t,d} - \Delta s_{t,d} \right\|_2 \tag{3}$$

and accept a synthesized sample only if $e_{t,d} \leq \tau$, where τ is calibrated on successful reference trajectories.

Joint Pose Verifier. We discard kinematically unsafe rollouts by evaluating joint limit and motion smoothness constraints in joint space. We train a joint pose predictor based on ResNet-50 [25] to estimate per-frame joint angles $q_t \in \mathbb{R}^7$ from the generated video, and compute joint velocities $\dot{q}_t$ and accelerations $\ddot{q}_t$ using finite differences. Thresholds are calibrated on successful reference trajectories. We reject a synthesized clip if there exists a time step t and joint

TABLE I: **Comparison to existing datasets.** We compare Dream2Fix with AHA [9] and REFLECT [12] in terms of dataset scale, input modalities, and label style. Dream2Fix pairs synthesized failure clips with structured labels that specify failure type and stage, together with executable corrections, enabling vision-language models to produce grounded failure diagnoses and executable corrections.

Source	Dream2Fix	AHA [9]	REFLECT [12]
			
Quantity	120K	49K	130
Input	Instruction: Put the lid on the pot. Provide RESULT and (if needed) a fix suggestion.	For the given sub-tasks, first determine it has succeed by choosing from ["yes", "no"] and then explain the reason why the current sub-tasks has failed.	RGB-D video frames, audio, robot proprioception
Label	RESULT=FAIL; TYPE=translation; STAGE=pre_grasp; FIX_DIR_X=-x; FIX_N_X=2; FIX_DIR_Y=+y; FIX_N_Y=3; The execution failed due to a translation misalignment before grasping. To fix it, nudge the end-effector in -x for 2 steps and in +y for 3 steps in the keyframe.	No, the robot gripper rotated with an incorrect roll angle.	At 03:00, the robot failed to properly place the pot filled with water on the stove burner. The failure was caused by the robot dropping something on the ground at 02:43, which might have been the pot or its contents. Failure timestep: 02:43

index j such that the predicted joint angle violates the joint limits, or the joint velocity exceeds a threshold, or the joint acceleration exceeds a threshold:

$$\exists t, j: \quad q_{t,j} \notin [q_j^{\min}, q_j^{\max}] \quad \text{or} \quad |\dot{q}_{t,j}| > \tau_v \quad \text{or} \quad |\ddot{q}_{t,j}| > \tau_a.$$

This verifier removes rollouts that appear visually plausible but imply unsafe or physically implausible joint dynamics.

Point-Tracking Verifier. We assess temporal coherence via point tracking [26]. Given M tracked points (uniformly sampled on a 10×10 grid in the initial frame), we obtain 2D trajectories $\{p_t^i\}_{t=1}^T$ in pixel space with visibility masks $\{m_t^i\}_{t=1}^T$. We compute four complementary metrics, including motion smoothness S_{smooth}, visibility stability S_{vis}, local topology stability S_{topo}, and global continuity S_{global}, and combine them into a total score S_{pt} for dataset curation. We clip all scores to the range $[0, 1]$. We additionally discard clips with insufficient tracking confidence. Let a denote the per-track second-order motion magnitude (discrete acceleration) computed from $\{p_t^i\}$, and let r_{spike} be the fraction of frames exhibiting spike-like transients. We define:

$$S_{\text{smooth}} = \exp\left(-\frac{Q_{0.95}(a)}{\tau_{\text{acc}}}\right) \cdot (1 - r_{\text{spike}}) \tag{4}$$

$$S_{\text{vis}} = 1 - \text{median}_i\left(\frac{1}{T-1}\sum_t \mathbf{1}\left[m_{t+1}^i \neq m_t^i\right]\right) \tag{5}$$

We form KNN edges in the first frame and track their pairwise distances over time. For an edge (i, j), let d_t^{ij} denote the distance at time t. We define:

$$u_t^{ij} = \frac{\left|d_t^{ij} - d_0^{ij}\right|}{d_0^{ij} + \epsilon}, \qquad S_{\text{topo}} = \exp\left(-\frac{\text{median}(u)}{\tau_{\text{topo}}}\right) \tag{6}$$

We fit an affine transform between adjacent frames and compute per-frame alignment errors (rmse) and jitter, defined as the temporal fluctuation of the estimated global affine parameters across consecutive frame pairs. Thus, we define:

$$\begin{aligned} S_{\text{global}} = 0.7 \exp\left(-\frac{Q_{0.9}(\text{rmse})}{\tau_{\text{rmse}}}\right) \\ + 0.3 \exp\left(-\frac{Q_{0.9}(\text{jitter})}{\tau_{\text{jitter}}}\right) \end{aligned} \tag{7}$$

Finally, we compute the total score:

$$\begin{aligned} S_{\text{pt}} = w_1 S_{\text{smooth}} + w_2 S_{\text{vis}} \\ + w_3 S_{\text{topo}} + w_4 S_{\text{global}}. \end{aligned} \tag{8}$$

3) Fix Suggestion Generation: For each verified failure rollout, we generate an actionable recovery as a structured text label that can be directly parsed by downstream modules. Specifically, the actionable recovery follows a fixed key–value schema, followed by a short natural language summary. The key–value fields specify actionable recovery in the trajectory, while the summary briefly describes the failure mode and the intended correction. Actionable recoveries are generated deterministically from the injected perturbation parameters used to generate the failure from success demonstrations, yielding consistent self-paired corrective supervision.

For gripper failures (delay_close, weak_close, force_open), the actionable recovery specifies the corrected gripper command and a temporal anchor indicating when the gripper state change should be executed. This is necessary because gripper failures may prevent the state change, making the keyframe ambiguous in the failed rollout. For translation failures, the actionable recovery specifies an axis-aligned correction direction and a discrete step count along each axis.

We avoid free-form recovery generation during dataset construction and instead adopt a predefined schema to keep the output space stable and unambiguous. Each dataset sample contains (i) a result label (success or fail), (ii) a failure mode, and (iii) an actionable recovery in a structured key–value format, optionally accompanied by a short natural language summary. The structured fields can be mapped to corrective control commands for the robot controllers.

C. Learn from Failure

1) VLM Finetuning: We follow a standard visual instruction-tuning paradigm to adapt Qwen3-VL-8B-Instruct to our failure reasoning and correction task. The model consists of a vision encoder, a multimodal projector that maps visual features into the language embedding space, and a transformer-based language model that generates text outputs autoregressively. Given a single-view manipulation video and the task instruction, the model is trained to output a structured response that includes the result label (success/fail), the failure type (delay_close, weak_close, force_open, translation), and an actionable correction in a key–value format with a natural language summary. We fine-tune the VLM on our Dream2Fix dataset constructed in Sec. III-B, and name the model as Dream2Fix-VLM. All components are initialized from the pre-trained checkpoint. During fine-tuning, we update the language model parameters and the multimodal projector parameters, while keeping other components frozen.

2) Closed-Loop Recovery for VLA: Dream2Fix enables real-world closed-loop recovery by grounding the VLM outputs into executable robot control primitives, thereby improving downstream VLA performance. We execute a base VLA policy on the real robot and record the execution trajectory. The record will first be downsampled and processed by Dream2Fix-VLM with the structured prompt, in order to reason about the failure. A deterministic mapping module is designed to parse the action recovery into robot control primitives, which are executed to correct the failed trajectory.

In practice, we select the keyframe and decide when to invoke the VLM using an event-aligned rule with a fixed action budget. If no closing attempt occurs within the budget, we terminate the rollout and extract a fixed-length clip from the final portion of the trajectory, then invoke the VLM to handle cases where the policy fails to initiate the gripper action. To execute the predicted actionable recovery, the mapping module applies a deterministic conversion from structured fields to control primitives. For translation failures, we convert the predicted axis-aligned adjustment into a corresponding translation delta in the robot control space.

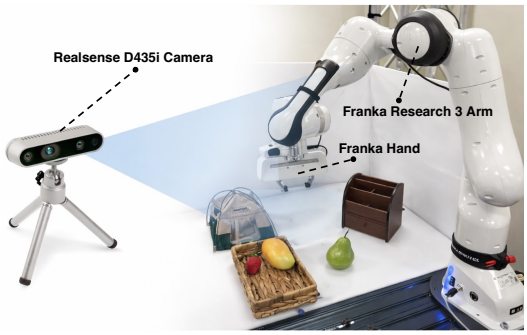


Fig. 3: **Real-world experimental workspace.** A Franka Research 3 arm with a Franka Hand gripper, and a fixed-view Intel RealSense D435i RGB-D camera are equipped for real-world evaluation.

TABLE II: **Generated data quality results.** We compare verifier metrics for ground-truth demonstrations (GT) and generated failure cases, where closer values indicate more similar distributions and higher-quality generated data.

Metric	GT	Generated
S_{smooth}	0.814	0.823
S_{vis}	0.999	0.998
S_{topo}	0.977	0.976
S_{global}	0.689	0.692
$\text{MAE}_{xyz}(\widehat{\Delta s}, \Delta s)$	0.008	0.015
$\text{MAE}_{rpy}(\widehat{\Delta s}, \Delta s)$	0.034	0.064
ω exceedance @p95	0.114	0.222
α exceedance @p95	0.339	0.462

For gripper failures, including missing, delayed, or weak closing, we issue an explicit gripper-close command at the predicted time and, when necessary, re-close or increase closing strength to ensure a firm grasp. After applying recovery, we replay the edited trajectory, record videos together with per-frame states and actions, and add recovered rollouts to the training database.

IV. Experiment

A. Experiment Setup

We evaluate Dream2Fix from two aspects: (i) VLM-based failure understanding and structured recovery prediction, and (ii) real-world closed-loop correction that grounds the predicted recovery into robot control to improve downstream robotic manipulation performance.

Benchmarks. We evaluate the VLMs on two test sets. The Dream2Fix test set is a split from our generated dataset, containing automatically generated failure records with structured labels for failure reasoning and recovery directives. The real-world failure test set is collected on the Franka Pandas robotic arm with a Realsense D435i camera and a Franka Hand (as shown in Fig 3), consisting of 100 trajectories across 10 manipulation tasks.

VLM Evaluation. We benchmark diverse state-of-the-art VLMs under a unified prompt and a structured output format. Each model takes the task instruction and an execution clip

as input, and predicts (a) the success/failure result, (b) the failure stage and type when failed, and (c) a structured correction. Metrics are defined below.

Metrics. We report six metrics that align with our evaluation scripts and tables: (i) *ROUGE-L* measures string-level overlap between predictions and references (for both the structured fields and the natural-language summary); (ii) *Cosine Similarity* measures semantic similarity using sentence embeddings; (iii) *Binary Success Accuracy* evaluates success/failure detection accuracy; (iv) *LLM-based Fuzzy Match* evaluates semantic correctness beyond surface-form overlap; we use Qwen3-30B-Instruct as the judge model and map its ratings (correct, partially_correct, incorrect) to scores $(1.0,\ 0.5,\ 0.0)$; (v) *Correction Accuracy* assigns partial credit to near-miss recovery directives on failure samples.

For translation failures, we define correction accuracy as:

$$\text{Acc} = \frac{s_{\text{type}} + s_{\text{stage}} + s_x + s_y}{4} \tag{9}$$

$$s_x = \mathbb{1}(\text{dir}_x^{gt} = \text{dir}_x^{pred}) \cdot \max(0,\ 1 - \tfrac{|n_x^{gt} - n_x^{pred}|}{\text{cap}}) \tag{10}$$

$$s_y = \mathbb{1}(\text{dir}_y^{gt} = \text{dir}_y^{pred}) \cdot \max(0,\ 1 - \tfrac{|n_y^{gt} - n_y^{pred}|}{\text{cap}}) \tag{11}$$

We represent the predicted translation correction along each axis $u \in \{x, y\}$ by a direction label $\text{dir}_u^{pred} \in \{+, -\}$ and a magnitude level n_u^{pred}. Likewise, dir_u^{gt} and n_u^{gt} are the ground-truth direction and magnitude level. The magnitude level n_u is a discretized bin index of the translation offset along axis u (larger n_u indicates a larger offset), and cap denotes the maximum bin difference tolerated when computing the accuracy. Here we set $\text{cap} = 3$ to tolerate up to three-bin deviation in the predicted translation magnitude.

For gripper failures, we define correction accuracy as:

$$\text{Acc} = \frac{s_{\text{type}} + s_{\text{stage}} + s_k}{3} \tag{12}$$

$$s_k = \mathbb{1}\big[|k^{gt} - k^{pred}| \le \delta_k\big] \tag{13}$$

Real-World Experiment. We evaluate real-world performance in two complementary settings. (i) offline VLM evaluation: we run VLM inference on the real-world failure test set, and report failure diagnosis quality and correction accuracy. (ii) real robot evaluation: we assess downstream performance in closed-loop robot execution in two scenarios. First, we execute the predicted corrections on a physical robot and report recovery success after applying the correction. Second, we deploy OpenVLA for manipulation and use our Dream2Fix-VLM to detect failures during execution and output real-time corrections when OpenVLA fails; we report the resulting end-to-end recovery rate.

B. Data Quality Verification.

Across our full validation mechanism, we remove 41.9% of synthesized rollouts and **retain 58.1%**, resulting in the final 120+k paired dataset used in our experiments. To characterize the distributional quality of synthesized failure cases, we compare verifier metrics between successful demonstrations and synthesized failures (as shown in Tab. II). Here s_t denotes the end-effector state, and $\Delta s_{t,d} = s_{t+d} - s_t$ denotes the corresponding end-effector state difference over an interval of length d.

For S_{smooth}, S_{vis}, S_{stop}, and S_{global}, we compute a sequence-level value for each sample and report the dataset mean. We also report the IDM consistency errors $\text{MAE}_{xyz}(\Delta\hat{s}, \Delta s)$ and $\text{MAE}_{rpy}(\Delta\hat{s}, \Delta s)$, where the IDM predicts an end-effector state difference $\Delta\hat{s}_{t,d} = f_\theta(I_t, I_{t+d})$ from two frames, and the reference difference $\Delta s_{t,d}$ is computed from recorded end-effector states used for data generation.

Because the IDM is a learned model, we first run it on successful demonstrations to obtain a reference error distribution, which reflects the prediction error of the IDM when the visual transition is real. We then apply the same IDM to synthesized failures and compute the same discrepancies between $\Delta\hat{s}_{t,d}$ inferred from the generated frames and the conditioning signal $\Delta s_{t,d}$. Comparing the resulting MAEs between successful demonstrations and synthesized failures isolates the inconsistency introduced by data generation beyond the IDM's error, and serves as a proxy for the physical and kinematic plausibility of the generated failures.

We report joint pose exceedance rates ω exceedance @p95 and α exceedance @p95 from the joint pose verifier. ω exceedance @p95 denotes the fraction of synthesized failures in which joint velocity exceeds a p95-calibrated velocity threshold, and α exceedance @p95 denotes the fraction in which joint acceleration exceeds a p95-calibrated acceleration threshold. The p95 velocity threshold τ_v and p95 acceleration threshold τ_a are calibrated on successful demonstrations by pooling $|\dot{q}_{t,j}|$ and $|\ddot{q}_{t,j}|$ over all time steps and joints, then setting τ_v and τ_a to the 95th percentile of the pooled distributions. A synthesized failure case contributes to ω exceedance @p95 if any joint at any time step exceeds τ_v, and it contributes to α exceedance @p95 if any joint at any time step exceeds τ_a.

Overall, synthesized failures remain close to successful demonstrations under our verifier metrics. The consistency scores (S_{smooth}, S_{vis}, S_{stop}, and S_{global}) stay in a comparable range across the two sets, suggesting that the synthesized failures preserve temporal smoothness, visual consistency, and coarse motion patterns observed in real executions. The IDM discrepancy measures and the ω exceedance @p95 and α exceedance @p95 rates fall within a moderate and controlled range, indicating that the synthesized failures maintain plausible end-effector transitions and joint motions relative to successful demonstrations. These results suggest that Dream2Fix produces failure cases with good visual fidelity and physical plausibility under our verifier metrics, supporting their use for learning and evaluating recovery.

C. Quantitative Results

We evaluate Dream2Fix in two settings: the Dream2Fix benchmark (test set) and a real-world benchmark under zero-shot manner. Table III summarizes results for diverse baselines, including open-source VLMs (LLaVA-NeXT-7B [27], LLaVA-NeXT-34B [27], Qwen2.5-VL-7B [28], and Qwen3-VL-8B [29]) and closed-source models (Gemini-3-Flash and GPT-4o). To make the comparison as fair as possible, all

Fig. 4: **Real world robot execution.** For each task, we show the initial failed execution (left) and the corrected execution (right). Across diverse tasks, the correction adjusts the action to recover from the failure under the same real-robot setup.

TABLE III: **Results on the Dream2Fix test set and real-world benchmarks.** We compare baselines with Dream2Fix-VLM (ours) under the same evaluation protocol on two settings: (i) Dream2Fix (Test Set) and (ii) real-world benchmark (Zero-Shot). Metrics include text similarity (ROUGE-L and cosine similarity), match-based success measures (BinSucc and fuzzy match), and correction accuracy (Acc.). ↑ indicates higher value is better.

Models	Dream2Fix Dataset (Test Set)					Real-World Benchmark (Zero-Shot)				
	ROUGE_L ↑	Cos. Sim. ↑	BinSucc(%) ↑	Fuzzy Match ↑	Acc. ↑	ROUGE_L ↑	Cos. Sim. ↑	BinSucc(%) ↑	Fuzzy Match ↑	Acc. ↑
LLaVA-NeXT-7B [27]	0.080	0.099	0.111	0.322	0.022	0.000	0.000	0.000	0.354	0.000
LLaVA-NeXT-34B [27]	0.130	0.127	0.153	0.453	0.044	0.090	0.090	0.300	0.128	0.022
Qwen2.5-VL-7B [28]	0.170	0.391	0.551	0.241	0.113	0.052	0.176	0.250	0.105	0.008
Qwen3-VL-8B [29]	0.248	0.386	0.567	0.248	0.143	0.061	0.478	0.930	0.167	0.183
Gemini-3-Flash [30]	0.406	0.518	0.679	0.496	0.179	0.467	0.589	**0.980**	0.250	0.374
GPT-4o	0.502	0.538	0.623	0.615	0.197	0.190	0.473	0.720	0.221	0.126
Dream2Fix-VLM (Ours)	**0.913**	**0.941**	**0.950**	**0.771**	**0.813**	**0.621**	**0.668**	0.820	**0.421**	**0.472**

models are prompted with the same structured output format that matches our annotation schema.

On the Dream2Fix benchmark, Dream2Fix-VLM achieves the best overall performance and delivers a substantial gain in correction accuracy. Specifically, Acc improves from 0.197 (GPT-4o) to 0.813, showing that Dream2Fix-VLM produces much more accurate recovery outputs rather than only identifying failures. Dream2Fix-VLM also attains the highest binary success rate, which is consistent with stronger recovery decisions beyond failure recognition.

For the real-world benchmark, Dream2Fix-VLM is evaluated in zero-shot, without fine-tuning on real-world data. Metrics are lower than on the Dream2Fix benchmark, since real-world data contains more varied appearances and execution conditions. Notably, several baselines achieve high binary success rates but still obtain low correction accuracy, suggesting that they can often judge success or failure while failing to produce correct recovery outputs. In contrast, Dream2Fix-VLM delivers the strongest correction accuracy on real-world data (Acc 0.472), supporting improved failure reasoning and correction capability under strict zero-shot evaluation, and demonstrating the usefulness of Dream2Fix data for learning corrective recovery in real-world settings.

D. Downstream Robotic Experiments

We evaluate downstream performance in real-world execution (as shown in Fig. 4) using two complementary settings. First, we test whether our VLM-predicted corrections can be executed on a real robot to recover from failures in the Real-World Benchmark. These cases reflect failures observed in real data, and the robot executes the predicted trajectory correction to attempt recovery. Second, we test recovery during deployment with OpenVLA [31]. We run OpenVLA on a set of manipulation tasks, collect its deployment failures, and then apply our predicted corrections to recover from these failures. In both settings, the Dream2Fix-VLM is trained only on Dream2Fix synthetic data and is applied to real-world data in a strict zero-shot manner. For each

TABLE IV: **Real-world results.** We report recovery rate (↑ is better) for two methods, Dream2Fix-VLM and Gemini3-Flash, evaluated on failures from our Real-World Benchmark and OpenVLA deployment.

Failure Source	Recovery Rate (%) ↑	
	Dream2Fix-VLM	Gemini3-Flash
Real-World Benchmark	46%	36%
OpenVLA Deployment	40%	30%

setting, we conduct 50 runs and report the recovery rate, computed over failure cases, where a failure is counted as recovered if executing the predicted correction leads to task success. Tab. IV summarizes the recovery rates for Real-World Benchmark and OpenVLA deployment failures.

V. Conclusions

We presented Dream2Fix, a data generation pipeline that synthesizes photorealistic manipulation failures from successful demonstrations and curates them with visual and physical verifier criteria. Using the resulting data, we trained Dream2Fix-VLM to produce actionable trajectory corrections beyond failure judgment. Across our validation mechanism, we remove 41.9% of synthesized rollouts and retain 58.1%, resulting in a final dataset of over 120k paired failure–correction samples. This retention ratio highlights that, while the current action-conditioned world model already serves as a strong controllable simulator for diverse failure generation, a substantial fraction of rollouts still deviates from realistic visual and physical behavior, which leaves room to further improve physical plausibility. Experiments on the Dream2Fix benchmark and real-world data show strong zero-shot correction accuracy, improving from 19.7% to 81.3%, and real-robot trials confirm that the predicted corrections can be executed to recover from failures. Future work will improve the action-conditioned dynamics of our robotic world model to produce more physically consistent rollouts, enabling higher-fidelity data augmentation at scale.

References

[1] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, "Palm-e: An embodied multimodal language model," *arXiv preprint arXiv:2303.03378*, 2023.

[2] B. Zitkovich *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Proceedings of The 7th Conference on Robot Learning*, vol. 229 of *Proceedings of Machine Learning Research*, pp. 2165–2183, PMLR, 2023.

[3] K. Bousmalis *et al.*, "Robocat: A self-improving generalist agent for robotic manipulation," *arXiv preprint arXiv:2306.11706*, 2023.

[4] Y. Ji, H. Tan, J. Shi, X. Hao, Y. Zhang, H. Zhang, P. Wang, M. Zhao, Y. Mu, P. An, *et al.*, "Robobrain: A unified brain model for robotic manipulation from abstract to concrete," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 1724–1734, 2025.

[5] W. Yuan, J. Duan, V. Blukis, W. Pumacay, R. Krishna, A. Murali, A. Mousavian, and D. Fox, "Robopoint: A vision-language model for spatial affordance prediction for robotics," *arXiv preprint arXiv:2406.10721*, 2024.

[6] T. Lee, A. Wagenmaker, K. Pertsch, P. Liang, S. Levine, and C. Finn, "Roboreward: General-purpose vision-language reward models for robotics," *arXiv preprint arXiv:2601.00675*, 2026.

[7] B. Peng, W. Zhang, L. Xu, Z. Qi, J. Zhang, H. Liu, W. Zeng, and X. Jin, "Reworld: Multi-dimensional reward modeling for embodied world models," *arXiv preprint arXiv:2601.12428*, 2026.

[8] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, *et al.*, "$\pi^*_{0.6}$: a vla that learns from experience," *arXiv preprint arXiv:2511.14759*, 2025.

[9] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo, "Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation," in *International Conference on Learning Representations (ICLR)*, 2025.

[10] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, and F. Shkurti, "SAFE: Multitask failure detection for vision-language-action models," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[11] P. Pacaud, R. G. Pinel, S. Chen, and C. Schmid, "Guardian: Detecting robotic planning and execution errors with vision-language models," in *Workshop on Making Sense of Data in Robotics: Composition, Curation, and Interpretability at Scale at CoRL 2025*, 2025.

[12] Z. Liu, A. Bahety, and S. Song, "Reflect: Summarizing robot experiences for failure explanation and correction," in *Conference on Robot Learning*, pp. 3468–3484, PMLR, 2023.

[13] C. Qi, X. Wang, S. Yong, S. Sheng, H. Mao, sriram srinivasan, M. Nambi, A. Zhang, and Y. Dattatreya, "Self-refining vision language model for robotic failure detection and reasoning," in *The Fourteenth International Conference on Learning Representations*, 2026.

[14] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, "Mimicgen: A data generation system for scalable robot learning using human demonstrations," in *Conference on Robot Learning*, pp. 1820–1864, PMLR, 2023.

[15] H. Xia, X. Li, Z. Li, Q. Ma, J. Xu, M.-Y. Liu, Y. Cui, T.-Y. Lin, W.-C. Ma, S. Wang, S. Song, and F. Wei, "Sage: Scalable agentic 3d scene generation for embodied ai," 2026.

[16] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan, "Robogen: towards unleashing infinite data for automated robot learning via generative simulation," in *Proceedings of the 41st International Conference on Machine Learning*, pp. 51936–51983, 2024.

[17] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, A. Mavalankar, C. Apps, Y. Aytar, and T. Rocktäschel, "Genie: Generative interactive environments," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235, pp. 4603–4623, PMLR, 2024.

[18] S. Zhou, Y. Du, J. Chen, Y. Li, D.-Y. Yeung, and C. Gan, "Robodreamer: Learning compositional world models for robot imagination," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235, pp. 61885–61896, PMLR, 2024.

[19] Y. Guo, L. X. Shi, J. Chen, and C. Finn, "Ctrl-world: A controllable generative world model for robot manipulation," *arXiv preprint arXiv:2510.10125*, 2025.

[20] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Mastering diverse control tasks through world models," *Nature*, 2025.

[21] J. Ye, R. Xue, B. Van Hoorick, P. Tokmakov, M. Z. Irshad, Y. Wang, and V. Guizilini, "Anchordream: Repurposing video diffusion for embodiment-aware robot data synthesis," *arXiv preprint arXiv:2512.11797*, 2025.

[22] W. Cai, I. Ponomarenko, J. Yuan, X. Li, W. Yang, H. Dong, and B. Zhao, "Spatialbot: Precise spatial understanding with vision language models," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9490–9498, IEEE, 2025.

[23] E. Zhou, J. An, C. Chi, Y. Han, S. Rong, C. Zhang, P. Wang, Z. Wang, T. Huang, L. Sheng, *et al.*, "Roborefer: Towards spatial referring with reasoning in vision-language models for robotics," *arXiv preprint arXiv:2506.04308*, 2025.

[24] W. Yuan, J. Duan, V. Blukis, W. Pumacay, R. Krishna, A. Murali, A. Mousavian, and D. Fox, "Robopoint: A vision-language model for spatial affordance prediction in robotics," in *Conference on Robot Learning*, pp. 4005–4020, PMLR, 2025.

[25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

[26] N. Karaev, I. Makarov, J. Wang, N. Neverova, A. Vedaldi, and C. Rupprecht, "Cotracker3: Simpler and better point tracking by pseudo-labelling real videos," in *Proc. arXiv:2410.11831*, 2024.

[27] B. Li, K. Zhang, H. Zhang, D. Guo, R. Zhang, F. Li, Y. Zhang, Z. Liu, and C. Li, "Llava-next: Stronger llms supercharge multimodal capabilities in the wild," May 2024.
[28] S. Bai, K. Chen, and X. L. et al., "Qwen2.5-vl technical report," 2025.
[29] A. Yang, A. Li, and B. Y. et al., "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.
[30] Google DeepMind, "Gemini 3." https://deepmind.google/models/gemini/, 2025. Accessed: 2026-02-28.
[31] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, *et al.*, "Openvla: An open-source vision-language-action model," in *8th Annual Conference on Robot Learning*.